Notebook 50: External Song Batch Testing with Full-Song Window Voting

Purpose

This notebook batch-tests external songs using the full-song window idea.

Notebook 49 is the formal FMA benchmark. Notebook 50 is a deployment-style workflow: each external song is split into 15-second windows, each window is scored by the trained audio CNN, and song-level Top-1, Top-3, and Top-5 predictions are created from the average window scores plus window vote counts.

Why Audio-Only Here?

External songs do not have trusted FMA structured metadata rows. The cleanest deployment test is therefore the audio branch with full-song window voting. The formal hybrid benchmark remains in Notebook 49.

Outputs

- Song-level summary CSV
- Candidate-level Top-N score CSV
- Window-level prediction CSV
- Error CSV
- JSON run summary

In [1]:

```
# ============================================================
# Cell 1: Imports
# ============================================================

import os
import json
import math
import time
import tempfile
import subprocess
import warnings
from pathlib import Path
from datetime import datetime

import numpy as np
import pandas as pd
```

```
try:
    import librosa
except Exception as e:
    raise ImportError("librosa is required. Install with: pip install librosa")
from e

try:
    import tensorflow as tf
    print("TensorFlow version:", tf.__version__)
except Exception as e:
    raise ImportError("TensorFlow could not be imported. Use the project .venv
kernel.") from e

warnings.filterwarnings("ignore")
print("Imports completed.")
```

```
TensorFlow version: 2.20.0
Imports completed.
```

In [2]:

```
# ============================================================
# Cell 2: User Settings
# ============================================================

PROJECT_ROOT = Path(r"E:\SCHOOL\Masters\Capstone_FMA_Project")
PROCESSED_DIR = PROJECT_ROOT / "data" / "processed"
MODELS_DIR = PROJECT_ROOT / "models"
METADATA_DIR = PROJECT_ROOT / "data" / "raw" / "metadata"

# Put songs in this folder, or change this path to your external-song folder.
EXTERNAL_AUDIO_DIR = PROJECT_ROOT / "data" / "external_audio_batch"
EXTERNAL_MANIFEST_XLSX_PATH = EXTERNAL_AUDIO_DIR / "external_audio_manifest.xlsx"
EXTERNAL_MANIFEST_CSV_PATH = EXTERNAL_AUDIO_DIR /
"external_audio_manifest_source.csv"
REQUIRE_MANIFEST_FOR_EVALUATION = True

OUTPUT_DIR = PROJECT_ROOT / "outputs" / "notebook50_external_song_batch_windowed"
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

AUDIO_EXTENSIONS = [".mp3", ".wav", ".flac", ".m4a", ".aac", ".ogg", ".wma",
".mp4", ".webm"]
MAX_FILES = None  # Example: 10 for a dry run, None for all files

# Full-song window settings.
WINDOW_SECONDS = 15
MAX_WINDOWS = 8
WINDOW_SELECTION_MODE = "evenly_spaced"  # "evenly_spaced" or "first_n"
LOOP_SHORT_AUDIO = True
LOOP_TARGET_SECONDS = WINDOW_SECONDS * min(MAX_WINDOWS, 4)

# Final audio/candidate settings.
STAGE1_THRESHOLD = 0.20
LOW_CONFIDENCE_THRESHOLD = 0.50
MIN_WINDOW_VOTES = 2
TOP_N_CANDIDATES = 15
```

```
# Audio CNN settings used during training.
SR = 22050
N_MELS = 64
N_FFT = 2048
HOP_LENGTH = 1024
MAX_FRAMES = int(np.ceil((WINDOW_SECONDS * SR) / HOP_LENGTH)) + 1

print("PROJECT_ROOT:", PROJECT_ROOT)
print("EXTERNAL_AUDIO_DIR:", EXTERNAL_AUDIO_DIR)
print("EXTERNAL_MANIFEST_XLSX_PATH:", EXTERNAL_MANIFEST_XLSX_PATH)
print("EXTERNAL_MANIFEST_CSV_PATH:", EXTERNAL_MANIFEST_CSV_PATH)
print("OUTPUT_DIR:", OUTPUT_DIR)
print("WINDOW_SECONDS:", WINDOW_SECONDS)
print("MAX_WINDOWS:", MAX_WINDOWS)
print("WINDOW_SELECTION_MODE:", WINDOW_SELECTION_MODE)
print("MIN_WINDOW_VOTES:", MIN_WINDOW_VOTES)
print("MAX_FRAMES:", MAX_FRAMES)
```

```
PROJECT_ROOT: E:\SCHOOL\Masters\Capstone_FMA_Project
EXTERNAL_AUDIO_DIR: E:\SCHOOL\Masters\Capstone_FMA_Project\data\external_audio_batch
EXTERNAL_MANIFEST_XLSX_PATH:
E:\SCHOOL\Masters\Capstone_FMA_Project\data\external_audio_batch\external_audio_manif
est.xlsx
EXTERNAL_MANIFEST_CSV_PATH:
E:\SCHOOL\Masters\Capstone_FMA_Project\data\external_audio_batch\external_audio_manif
est_source.csv
OUTPUT_DIR:
E:\SCHOOL\Masters\Capstone_FMA_Project\outputs\notebook50_external_song_batch_windowe
d
WINDOW_SECONDS: 15
MAX_WINDOWS: 8
WINDOW_SELECTION_MODE: evenly_spaced
MIN_WINDOW_VOTES: 2
MAX_FRAMES: 324
```

In [3]:

```
# ============================================================
# Cell 3: Load Frozen Audio Model and Label Metadata
# ============================================================

audio_model = tf.keras.models.load_model(
    MODELS_DIR / "audio_multilabel_candidate150_expanded_final.keras"
)

candidate_label_cols = np.load(
    PROCESSED_DIR / "hybrid_multilabel_candidate150_expanded_label_columns.npy",
    allow_pickle=True,
)
candidate_label_cols = [str(x) for x in candidate_label_cols.tolist()]
candidate_label_ids = [int(col.replace("genre_", "")) for col in
candidate_label_cols]

genres_df = pd.read_csv(METADATA_DIR / "genres.csv")
if "genre_id" not in genres_df.columns:
```

```
    genres_df = genres_df.reset_index().rename(columns={"index": "genre_id"})
genres_df["genre_id"] = genres_df["genre_id"].astype(int)

id_to_name = dict(zip(genres_df["genre_id"], genres_df["title"].astype(str)))
id_to_parent = dict(zip(genres_df["genre_id"], genres_df.get("parent",
pd.Series([0] * len(genres_df))).fillna(0).astype(int)))
id_to_top = dict(zip(genres_df["genre_id"], genres_df.get("top_level",
genres_df["genre_id"]).fillna(genres_df["genre_id"]).astype(int)))

with open(PROCESSED_DIR /
"audio_multilabel_candidate150_expanded_best_threshold.txt", "r") as f:
    audio_best_threshold = float(f.read().strip())

STAGE1_THRESHOLD = audio_best_threshold

print("Audio model input shape:", audio_model.input_shape)
print("Audio model output shape:", audio_model.output_shape)
print("Candidate labels:", len(candidate_label_cols))
print("Audio best threshold:", STAGE1_THRESHOLD)
print("First labels:", candidate_label_cols[:5])
```

```
Audio model input shape: (None, 64, 324, 1)
Audio model output shape: (None, 150)
Candidate labels: 150
Audio best threshold: 0.2
First labels: ['genre_1', 'genre_2', 'genre_3', 'genre_4', 'genre_5']
```

In [4]:

```
# ============================================================
# Cell 4: Helper Functions
# ============================================================

def timestamp_now():
    return datetime.now().strftime("%Y-%m-%d %H:%M:%S")


def genre_name_from_col(label_col):
    gid = int(str(label_col).replace("genre_", ""))
    return id_to_name.get(gid, str(gid))


def confidence_level(score):
    if score >= 0.70:
        return "High"
    if score >= 0.50:
        return "Moderate"
    if score >= 0.30:
        return "Low-to-moderate"
    return "Low"


def classify_relationship_to_main(genre_id, main_genre_id):
    if genre_id == main_genre_id:
        return "Primary genre"
```

```
    g_parent = id_to_parent.get(genre_id)
    g_root = id_to_top.get(genre_id)
    main_parent = id_to_parent.get(main_genre_id)
    main_root = id_to_top.get(main_genre_id)

    if g_parent == main_genre_id or g_root == main_genre_id:
        return "Subgenre/child under primary genre"
    if main_parent == genre_id or main_root == genre_id:
        return "Broader parent of primary genre"
    if g_root is not None and main_root is not None and g_root == main_root:
        return "Related genre in same top-level family"
    return "Secondary associated genre"


def load_audio_robust(file_path, sr=SR):
    try:
        y, sr_loaded = librosa.load(file_path, sr=sr, mono=True)
        if y is None or len(y) == 0:
            raise ValueError(f"Loaded audio is empty: {file_path}")
        return y, sr_loaded
    except Exception as first_error:
        ext = Path(file_path).suffix.lower()
        fallback_exts = {".mp4", ".m4a", ".aac", ".mov", ".3gp", ".webm", ".wma"}
        if ext not in fallback_exts:
            raise RuntimeError(f"Could not load audio file: {file_path}\nOriginal
error: {first_error}")

        tmp_wav = None
        try:
            with tempfile.NamedTemporaryFile(suffix=".wav", delete=False) as tmp:
                tmp_wav = tmp.name
            cmd = ["ffmpeg", "-y", "-i", str(file_path), "-ac", "1", "-ar",
str(sr), tmp_wav]
            result = subprocess.run(cmd, stdout=subprocess.PIPE,
stderr=subprocess.PIPE, text=True)
            if result.returncode != 0:
                raise RuntimeError(result.stderr)
            y, sr_loaded = librosa.load(tmp_wav, sr=sr, mono=True)
            if y is None or len(y) == 0:
                raise ValueError(f"Converted audio is empty: {file_path}")
            return y, sr_loaded
        finally:
            if tmp_wav:
                try:
                    os.remove(tmp_wav)
                except Exception:
                    pass


def loop_audio_if_needed(y, sr, target_seconds=LOOP_TARGET_SECONDS):
    target_len = int(target_seconds * sr)
    if not LOOP_SHORT_AUDIO or len(y) >= target_len:
        return y.astype(np.float32), False
    repeats = math.ceil(target_len / max(len(y), 1))
    return np.tile(y, repeats)[:target_len].astype(np.float32), True
```

```
def get_window_start_samples(y, sr=SR, window_seconds=WINDOW_SECONDS,
max_windows=MAX_WINDOWS, mode=WINDOW_SELECTION_MODE):
    total_len = len(y)
    window_len = int(window_seconds * sr)
    if total_len <= window_len or max_windows <= 1:
        return [0]

    max_start = total_len - window_len
    if mode == "first_n":
        starts = []
        current = 0
        while current <= max_start and len(starts) < max_windows:
            starts.append(int(current))
            current += window_len
        return starts if starts else [0]

    if mode == "evenly_spaced":
        n_windows = min(max_windows, max(2, math.ceil(total_len / window_len)))
        starts = np.linspace(0, max_start, num=n_windows)
        return sorted(list(dict.fromkeys([int(x) for x in starts])))[:max_windows]

    raise ValueError("WINDOW_SELECTION_MODE must be 'evenly_spaced' or 'first_n'.")


def extract_window(y, start_sample, sr=SR, window_seconds=WINDOW_SECONDS):
    window_len = int(window_seconds * sr)
    segment = y[start_sample:start_sample + window_len]
    if len(segment) < window_len:
        segment = np.pad(segment, (0, window_len - len(segment)), mode="constant")
    return segment.astype(np.float32)


def build_mel_input(y_segment, sr=SR, n_mels=N_MELS, n_fft=N_FFT,
hop_length=HOP_LENGTH, max_frames=MAX_FRAMES):
    mel = librosa.feature.melspectrogram(
        y=y_segment,
        sr=sr,
        n_fft=n_fft,
        hop_length=hop_length,
        n_mels=n_mels,
    )
    mel_db = librosa.power_to_db(mel, ref=np.max)
    mel_db = np.clip((mel_db + 80.0) / 80.0, 0.0, 1.0)

    if mel_db.shape[1] < max_frames:
        mel_db = np.pad(mel_db, ((0, 0), (0, max_frames - mel_db.shape[1])),
mode="constant")
    else:
        mel_db = mel_db[:, :max_frames]

    return mel_db.astype(np.float32)[None, :, :, None]


def split_pipe_values(value):
    if value is None or (isinstance(value, float) and np.isnan(value)):
        return []
    parts = [str(x).strip() for x in str(value).split("|")]
```

```
    return [x for x in parts if x and x.lower() not in {"nan", "none"}]


def normalize_manifest_key(value):
    if value is None or (isinstance(value, float) and np.isnan(value)):
        return ""
    return str(value).replace("\\", "/").strip().lower()


def load_external_manifest(xlsx_path=EXTERNAL_MANIFEST_XLSX_PATH,
csv_path=EXTERNAL_MANIFEST_CSV_PATH):
    manifest_path = None
    manifest_df = pd.DataFrame()

    # Prefer the CSV for notebook execution because it avoids optional Excel-engine
dependencies.
    # The .xlsx workbook remains the human-readable manifest artifact.
    if Path(csv_path).exists():
        manifest_df = pd.read_csv(csv_path)
        manifest_path = Path(csv_path)

    if manifest_df.empty and Path(xlsx_path).exists():
        try:
            manifest_df = pd.read_excel(xlsx_path, sheet_name="Manifest")
            manifest_path = Path(xlsx_path)
        except Exception as e:
            print("Could not read Excel manifest. Install openpyxl or use the
manifest CSV.")
            print("Excel read error:", e)

    if manifest_df.empty:
        print("No external manifest found. Notebook 50 will still predict audio
files, but Top-k correctness cannot be evaluated.")
        return pd.DataFrame(), manifest_path

    required_cols = [
        "file_name",
        "relative_path",
        "local_path",
        "song_title",
        "artist",
        "expected_primary_genre",
        "acceptable_genres",
        "acceptable_labels",
        "source",
        "source_item_url",
        "source_file_url",
        "license_url",
    ]
    for col in required_cols:
        if col not in manifest_df.columns:
            manifest_df[col] = ""

    manifest_df["file_name"] = manifest_df["file_name"].fillna("").astype(str)
    manifest_df["relative_path"] =
manifest_df["relative_path"].fillna("").astype(str)
    manifest_df["local_path"] = manifest_df["local_path"].fillna("").astype(str)
```

```
    manifest_df["file_name_key"] = manifest_df["file_name"].str.strip().str.lower()
    manifest_df["relative_path_key"] =
manifest_df["relative_path"].apply(normalize_manifest_key)
    manifest_df["local_path_key"] =
manifest_df["local_path"].apply(normalize_manifest_key)

    print("External manifest loaded:", manifest_path)
    print("Manifest rows:", len(manifest_df))
    if "expected_primary_genre" in manifest_df.columns:
        print("Manifest genre counts:")

display(manifest_df["expected_primary_genre"].value_counts().rename_axis("genre").r
eset_index(name="songs"))

    return manifest_df, manifest_path


def resolve_manifest_audio_path(row):
    candidates = []

    local_path = str(row.get("local_path", "")).strip()
    relative_path = str(row.get("relative_path", "")).strip()
    file_name = str(row.get("file_name", "")).strip()

    if local_path and local_path.lower() not in {"nan", "none"}:
        candidates.append(Path(local_path))
    if relative_path and relative_path.lower() not in {"nan", "none"}:
        candidates.append(EXTERNAL_AUDIO_DIR / Path(relative_path.replace("/",
os.sep)))
    if file_name and file_name.lower() not in {"nan", "none"}:
        candidates.append(EXTERNAL_AUDIO_DIR / file_name)

    for candidate in candidates:
        if candidate.exists():
            return candidate

    if file_name and file_name.lower() not in {"nan", "none"}:
        matches = list(EXTERNAL_AUDIO_DIR.rglob(file_name))
        if matches:
            return matches[0]

    return None

print("Helper functions ready.")
```

```
Helper functions ready.
```

In [5]:

```
# ============================================================
# Cell 5: Discover External Audio Files
# ============================================================

EXTERNAL_AUDIO_DIR.mkdir(parents=True, exist_ok=True)

manifest_df, manifest_path = load_external_manifest()
```

```
audio_manifest_df = pd.DataFrame()
missing_manifest_files = []

audio_files = []

if len(manifest_df):
    resolved_paths = []
    for _, row in manifest_df.iterrows():
        resolved = resolve_manifest_audio_path(row)
        if resolved is None:
            missing_manifest_files.append({
                "file_name": row.get("file_name", ""),
                "relative_path": row.get("relative_path", ""),
                "local_path": row.get("local_path", ""),
                "expected_primary_genre": row.get("expected_primary_genre", ""),
            })
            resolved_paths.append("")
        else:
            resolved_paths.append(str(resolved))
            audio_files.append(resolved)

    audio_manifest_df = manifest_df.copy()
    audio_manifest_df["resolved_audio_path"] = resolved_paths
    audio_manifest_df =
audio_manifest_df[audio_manifest_df["resolved_audio_path"].astype(str).str.len() >
0].copy()

    if MAX_FILES is not None:
        audio_files = audio_files[:int(MAX_FILES)]
        keep_paths = {str(p) for p in audio_files}
        audio_manifest_df =
audio_manifest_df[audio_manifest_df["resolved_audio_path"].isin(keep_paths)].copy()
else:
    for ext in AUDIO_EXTENSIONS:
        audio_files.extend(EXTERNAL_AUDIO_DIR.rglob(f"*{ext}"))
        audio_files.extend(EXTERNAL_AUDIO_DIR.rglob(f"*{ext.upper()}"))

    audio_files = sorted(set(audio_files))

    if MAX_FILES is not None:
        audio_files = audio_files[:int(MAX_FILES)]

print("External audio folder:", EXTERNAL_AUDIO_DIR)
print("Manifest used:", manifest_path)
print("Audio files selected:", len(audio_files))
print("Missing manifest files:", len(missing_manifest_files))

if missing_manifest_files:
    display(pd.DataFrame(missing_manifest_files).head(20))
    if REQUIRE_MANIFEST_FOR_EVALUATION:
        print("Warning: some manifest rows did not resolve to local audio files.")

if len(audio_files) == 0:
    print("Put audio files in EXTERNAL_AUDIO_DIR, then rerun from this cell.")
else:
```

```
    for p in audio_files[:20]:
        print("-", p.name)
```

```
External manifest loaded:
E:\SCHOOL\Masters\Capstone_FMA_Project\data\external_audio_batch\external_audio_manif
est_source.csv
Manifest rows: 60
Manifest genre counts:
```

	genre	songs
0	Rock	10
1	Electronic	10
2	Experimental	10
3	Folk	10
4	Hip-Hop	10
5	Pop	10

```
External audio folder:
E:\SCHOOL\Masters\Capstone_FMA_Project\data\external_audio_batch
Manifest used:
E:\SCHOOL\Masters\Capstone_FMA_Project\data\external_audio_batch\external_audio_manif
est_source.csv
Audio files selected: 60
Missing manifest files: 0
- Rock__ca200_cjazz__503._Andrij_Orel_-_Ringo_Quartz.mp3
-
Rock__afm004_allmyfaults_neonoir__The_course_of_true_love_never_did_run_smooth__my_de
arest.mp3
- Rock__Lethargie.LP__Intro___Im_Herbst.mp3
- Rock__Bostaurus.Demo__The_Sky_Full_of_Shadows.mp3
- Rock__BS450014__Last_Addiction.mp3
- Rock__csr020__Set_The_Controls_For_the_Heart_of_the_Sun.mp3
- Rock__afm020_noctiflora_0407__Neorama.mp3
- Rock__SCL160__Break_the_Lies__Gnothi_Seauton_Remix.mp3
- Rock__freemusiccharts.songs2012__2012-01-H._Zombie-Taxi_Driver.mp3
- Rock__diymAR06__10-Derek_Clegg-Judy.flac
- Electronic__NS050__Lake.mp3
- Electronic__mtk140__Csanca.mp3
- Electronic__foot149__El_And_Eu.mp3
- Electronic__nullbomb__spindrumlikeaboom.mp3
- Electronic__TFR100-VA-TornFleshRecordsPresentsVestigialSickness__36_-
_Vulgar_Disease_-_Every_Noise_Has_Its_TORN.mp3
- Electronic__MIXG031__Vozduh.mp3
- Electronic__pn037__Sabrina.mp3
- Electronic__mtcomp003__latatoo__secede_s_another_day_wasted_mix.mp3
- Electronic__ffs104__The_Leather.mp3
- Electronic__foot018__Baiao_Lo-fi.mp3
```

In [6]:

```
# ============================================================
# Cell 6: Predict One Song with Full-Song Window Voting
# ============================================================

def predict_external_song(audio_path):
    audio_path = Path(audio_path)
    song_name = audio_path.stem
    file_name = audio_path.name
    try:
        relative_audio_path =
str(audio_path.resolve().relative_to(EXTERNAL_AUDIO_DIR.resolve())).replace("\\",
"/")
    except Exception:
        relative_audio_path = file_name

    y_original, sr_loaded = load_audio_robust(audio_path, sr=SR)
    original_duration = len(y_original) / sr_loaded

    y_for_inference, audio_was_looped = loop_audio_if_needed(
        y_original,
        sr=sr_loaded,
        target_seconds=LOOP_TARGET_SECONDS,
    )
    inference_duration = len(y_for_inference) / sr_loaded

    starts = get_window_start_samples(
        y_for_inference,
        sr=sr_loaded,
        window_seconds=WINDOW_SECONDS,
        max_windows=MAX_WINDOWS,
        mode=WINDOW_SELECTION_MODE,
    )

    window_rows = []
    window_prob_list = []

    for window_index, start_sample in enumerate(starts):
        segment = extract_window(y_for_inference, start_sample, sr=sr_loaded,
window_seconds=WINDOW_SECONDS)
        X_audio = build_mel_input(
            segment,
            sr=sr_loaded,
            n_mels=N_MELS,
            n_fft=N_FFT,
            hop_length=HOP_LENGTH,
            max_frames=MAX_FRAMES,
        )
        probs = audio_model.predict(X_audio, verbose=0)[0]
        window_prob_list.append(probs)

        top_order = np.argsort(probs)[::-1]
        top1_idx = int(top_order[0])
        top3_idx = top_order[:3]
        top5_idx = top_order[:5]

        window_rows.append({
```

```
            "song_name": song_name,
            "audio_path": str(audio_path),
            "window_index": int(window_index),
            "start_sample": int(start_sample),
            "start_second": round(start_sample / sr_loaded, 2),
            "end_second": round((start_sample / sr_loaded) + WINDOW_SECONDS, 2),
            "top1_label": candidate_label_cols[top1_idx],
            "top1_genre_name": genre_name_from_col(candidate_label_cols[top1_idx]),
            "top1_score": float(probs[top1_idx]),
            "top3_labels": "|".join(candidate_label_cols[j] for j in top3_idx),
            "top3_genre_names":
"|".join(genre_name_from_col(candidate_label_cols[j]) for j in top3_idx),
            "top5_labels": "|".join(candidate_label_cols[j] for j in top5_idx),
            "top5_genre_names":
"|".join(genre_name_from_col(candidate_label_cols[j]) for j in top5_idx),
        })

    window_probs = np.vstack(window_prob_list)
    mean_scores = window_probs.mean(axis=0)
    max_scores = window_probs.max(axis=0)
    threshold_matrix = (window_probs >= STAGE1_THRESHOLD).astype(int)

    top1_per_window = np.argmax(window_probs, axis=1)
    top3_per_window = np.argsort(-window_probs, axis=1)[:, :3]
    top5_per_window = np.argsort(-window_probs, axis=1)[:, :5]

    required_window_votes = min(MIN_WINDOW_VOTES, len(starts))
    candidate_rows = []

    for j, label_col in enumerate(candidate_label_cols):
        genre_id = int(label_col.replace("genre_", ""))
        genre_name = genre_name_from_col(label_col)
        threshold_vote_count = int(threshold_matrix[:, j].sum())
        top1_vote_count = int(np.sum(top1_per_window == j))
        top3_vote_count = int(np.sum([j in row for row in top3_per_window]))
        top5_vote_count = int(np.sum([j in row for row in top5_per_window]))
        stable_prediction = int(
            (mean_scores[j] >= STAGE1_THRESHOLD)
            and (threshold_vote_count >= required_window_votes)
        )

        candidate_rows.append({
            "song_name": song_name,
            "audio_path": str(audio_path),
            "genre_id": genre_id,
            "label": label_col,
            "genre_name": genre_name,
            "mean_audio_score": float(mean_scores[j]),
            "max_audio_score": float(max_scores[j]),
            "confidence_level": confidence_level(float(mean_scores[j])),
            "threshold_vote_count": threshold_vote_count,
            "threshold_vote_rate": threshold_vote_count / len(starts),
            "top1_window_vote_count": top1_vote_count,
            "top3_window_vote_count": top3_vote_count,
            "top5_window_vote_count": top5_vote_count,
            "required_window_votes": required_window_votes,
            "stable_prediction": stable_prediction,
```

```
        })

    candidate_df = pd.DataFrame(candidate_rows).sort_values(
        ["mean_audio_score", "threshold_vote_count", "top3_window_vote_count"],
        ascending=[False, False, False],
    ).reset_index(drop=True)

    main_row = candidate_df.iloc[0].copy()
    main_genre_id = int(main_row["genre_id"])
    candidate_df["relationship_to_main"] = candidate_df["genre_id"].apply(
        lambda gid: classify_relationship_to_main(int(gid), main_genre_id)
    )

    top1 = candidate_df.head(1)
    top3 = candidate_df.head(3)
    top5 = candidate_df.head(5)
    predicted_df = candidate_df[candidate_df["stable_prediction"] == 1].copy()

    song_row = {
        "song_name": song_name,
        "file_name": file_name,
        "relative_path": relative_audio_path,
        "audio_path": str(audio_path),
        "status": "ok",
        "error": "",
        "original_duration_seconds": round(original_duration, 2),
        "inference_duration_seconds": round(inference_duration, 2),
        "audio_was_looped": bool(audio_was_looped),
        "windows_used": int(len(starts)),
        "window_seconds": WINDOW_SECONDS,
        "main_label": str(main_row["label"]),
        "main_genre_id": main_genre_id,
        "main_genre_name": str(main_row["genre_name"]),
        "main_score": float(main_row["mean_audio_score"]),
        "main_confidence_level":
confidence_level(float(main_row["mean_audio_score"])),
        "main_threshold_vote_count": int(main_row["threshold_vote_count"]),
        "main_top1_window_vote_count": int(main_row["top1_window_vote_count"]),
        "main_top3_window_vote_count": int(main_row["top3_window_vote_count"]),
        "low_confidence": bool(float(main_row["mean_audio_score"]) <
LOW_CONFIDENCE_THRESHOLD),
        "top1_labels": "|".join(top1["label"].astype(str).tolist()),
        "top1_genres": "|".join(top1["genre_name"].astype(str).tolist()),
        "top1_scores": "|".join(f"{x:.6f}" for x in
top1["mean_audio_score"].tolist()),
        "top3_labels": "|".join(top3["label"].astype(str).tolist()),
        "top3_genres": "|".join(top3["genre_name"].astype(str).tolist()),
        "top3_scores": "|".join(f"{x:.6f}" for x in
top3["mean_audio_score"].tolist()),
        "top5_labels": "|".join(top5["label"].astype(str).tolist()),
        "top5_genres": "|".join(top5["genre_name"].astype(str).tolist()),
        "top5_scores": "|".join(f"{x:.6f}" for x in
top5["mean_audio_score"].tolist()),
        "stable_predicted_labels":
"|".join(predicted_df["label"].astype(str).tolist()),
        "stable_predicted_genres":
"|".join(predicted_df["genre_name"].astype(str).tolist()),
```

```
    }

    return song_row, candidate_df.head(TOP_N_CANDIDATES), pd.DataFrame(window_rows)

print("Prediction function ready.")
```

```
Prediction function ready.
```

In [7]:

```
# ============================================================
# Cell 7: Batch Prediction Loop
# ============================================================

song_rows = []
candidate_tables = []
window_tables = []
error_rows = []

start_time = time.time()

for idx, audio_path in enumerate(audio_files, start=1):
    print(f"[{idx}/{len(audio_files)}] Processing: {audio_path.name}")
    t0 = time.time()

    try:
        song_row, candidate_df, window_df = predict_external_song(audio_path)
        song_row["seconds"] = time.time() - t0
        song_rows.append(song_row)
        candidate_tables.append(candidate_df)
        window_tables.append(window_df)
    except Exception as e:
        error_row = {
            "song_name": Path(audio_path).stem,
            "file_name": Path(audio_path).name,
            "relative_path": str(Path(audio_path).name),
            "audio_path": str(audio_path),
            "status": "error",
            "error": str(e),
            "seconds": time.time() - t0,
        }
        song_rows.append(error_row)
        error_rows.append(error_row)
        print("  ERROR:", e)

song_summary_df = pd.DataFrame(song_rows)
candidate_results_df = pd.concat(candidate_tables, ignore_index=True) if
candidate_tables else pd.DataFrame()
window_results_df = pd.concat(window_tables, ignore_index=True) if window_tables
else pd.DataFrame()
error_df = pd.DataFrame(error_rows)

elapsed = time.time() - start_time

print("Batch completed.")
print("Songs attempted:", len(audio_files))
```

```
print("Successful songs:", int((song_summary_df.get("status", pd.Series(dtype=str))
== "ok").sum()) if len(song_summary_df) else 0)
print("Errors:", len(error_df))
print("Elapsed minutes:", round(elapsed / 60, 2))

display(song_summary_df.head(20))
```

```
[1/60] Processing: Rock__ca200_cjazz__503._Andrij_Orel_-_Ringo_Quartz.mp3
[2/60] Processing:
Rock__afm004_allmyfaults_neonoir__The_course_of_true_love_never_did_run_smooth__my_de
arest.mp3
[3/60] Processing: Rock__Lethargie.LP__Intro___Im_Herbst.mp3
[4/60] Processing: Rock__Bostaurus.Demo__The_Sky_Full_of_Shadows.mp3
[5/60] Processing: Rock__BS450014__Last_Addiction.mp3
[6/60] Processing: Rock__csr020__Set_The_Controls_For_the_Heart_of_the_Sun.mp3
[7/60] Processing: Rock__afm020_noctiflora_0407__Neorama.mp3
[8/60] Processing: Rock__SCL160__Break_the_Lies__Gnothi_Seauton_Remix.mp3
[9/60] Processing: Rock__freemusiccharts.songs2012__2012-01-H._Zombie-Taxi_Driver.mp3
[10/60] Processing: Rock__diymAR06__10-Derek_Clegg-Judy.flac
[11/60] Processing: Electronic__NS050__Lake.mp3
[12/60] Processing: Electronic__mtk140__Csanca.mp3
[13/60] Processing: Electronic__foot149__El_And_Eu.mp3
[14/60] Processing: Electronic__nullbomb__spindrumlikeaboom.mp3
[15/60] Processing: Electronic__TFR100-VA-
TornFleshRecordsPresentsVestigialSickness__36_-_Vulgar_Disease_-
_Every_Noise_Has_Its_TORN.mp3
[16/60] Processing: Electronic__MIXG031__Vozduh.mp3
[17/60] Processing: Electronic__pn037__Sabrina.mp3
[18/60] Processing:
Electronic__mtcomp003__latatoo__secede_s_another_day_wasted_mix.mp3
[19/60] Processing: Electronic__ffs104__The_Leather.mp3
[20/60] Processing: Electronic__foot018__Baiao_Lo-fi.mp3
[21/60] Processing: Experimental__pcr089EmilDavydov-Sketches__Sketch_No.3.mp3
[22/60] Processing: Experimental__PoodleplayArkestra-
TheoryOfColour__Theory_Of_Colour.mp3
[23/60] Processing: Experimental__ca446_opa__04_-_Russian_State_Museum_s_Lottery.mp3
[24/60] Processing: Experimental__AlbumInADayVol4__Summer_Breeze.mp3
[25/60] Processing: Experimental__ca350_g__sh-dah.mp3
[26/60] Processing: Experimental__mz001_Mademoi_Selle__Himitsu.mp3
[27/60] Processing: Experimental__tpd074__08_Grow_Up_Lamestream_Turd_Burglers.ogg
[28/60] Processing: Experimental__ca492_ai__Beautiful_Distraction.mp3
[29/60] Processing: Experimental__ca300_ca__206_-_Mons_Jacet_-_for_spring.mp3
[30/60] Processing: Experimental__csr008__Say_Man.mp3
[31/60] Processing: Folk__lbn016Plusplus-gameOver__04_Plusplus_-_Broken_Doors.mp3
[32/60] Processing: Folk__lbn007NickRivera-happySongIsAHappySong__01_ReneeLuise.mp3
[33/60] Processing: Folk__lbn017OldSplendifolia-utterlyHeartbreaking__Two_Eagles.mp3
[34/60] Processing: Folk__EntertainmentForTheBraindead-Roadkillaaahh.009__Relapse.mp3
[35/60] Processing: Folk__lclcb05DelgarmaDerAndrafolkWorldChanson__LCLCB05_-
_Delgarma_-_Der_Andra_-_05_-_You_can_stay.mp3
[36/60] Processing: Folk__pm007__The_Treeplanting_Song.mp3
[37/60] Processing: Folk__pantoufle_tagada__I_Hope_You_re_Happy_With_Me.mp3
[38/60] Processing: Folk__DDW001__02_-_Built_From_Sticks_-_Here_comes_the_circus.flac
[39/60] Processing: Folk__ca201_meg__My_Orchard_Fire.mp3
[40/60] Processing: Folk__ekleipsi23__system_1.mp3
[41/60] Processing: Hip-Hop__DWK123__Soul_Key.mp3
```

```
[42/60] Processing: Hip-Hop__DWK031__Aydio_-_01_-_Track.mp3
[43/60] Processing: Hip-Hop__dystopiaq029__Times_re_Changin.mp3
[44/60] Processing: Hip-Hop__DWK149__Boogieman_Penthouse.mp3
[45/60] Processing: Hip-Hop__mia049__lifelong_fiction.mp3
[46/60] Processing: Hip-Hop__DWK217__Morning_Mist.mp3
[47/60] Processing: Hip-Hop__DWK301__Soul_Blue_Tango.mp3
[48/60] Processing: Hip-Hop__DWK127__The_Werewolf.mp3
[49/60] Processing: Hip-Hop__DWK315__Time2Spare.mp3
[50/60] Processing: Hip-Hop__DWK155__Eazy_Livin.mp3
[51/60] Processing: Pop__WkBw0034__01_-_Monochromatic_-_Immobility.mp3
[52/60] Processing: Pop__BSOG0008__05_-_Doing_Time.mp3
[53/60] Processing: Pop__starfrosch-mostwanted__Closer_to_You__feat._SHA.mp3
[54/60] Processing: Pop__SCL129__Heaven_In_Your_Mouth.mp3
[55/60] Processing: Pop__csr002__Falling.mp3
[56/60] Processing: Pop__12rec.026__The_Virus_Left_Behind.mp3
[57/60] Processing: Pop__iatmp3001__Blue_skies.mp3
[58/60] Processing: Pop__BSCOMP0006__09_-_Universal_Materials_-
_This_is_the_Macrophage.mp3
[59/60] Processing: Pop__csr079__Runaway.mp3
[60/60] Processing: Pop__mmm016__Mark_Robinson_-_Cape_May.flac
Batch completed.
Songs attempted: 60
Successful songs: 60
Errors: 0
Elapsed minutes: 1.59
```

	song_name	file
0	Rock__ca200_cjazz__503._Andrij_Orel_-_Ringo_Qu...	Rock__ca200_cjazz__503._Andrij_Orel_-_Ring
1	Rock__afm004_allmyfaults_neonoir__The_course_o...	Rock__afm004_allmyfaults_neonoir__The_cou
2	Rock__Lethargie.LP__Intro___Im_Herbst	Rock__Lethargie.LP__Intro___Im_Herb
3	Rock__Bostaurus.Demo__The_Sky_Full_of_Shadows	Rock__Bostaurus.Demo__The_Sky_Full_of_Shadov
4	Rock__BS450014__Last_Addiction	Rock__BS450014__Last_Addictio
5	Rock__csr020__Set_The_Controls_For_the_Heart_o...	Rock__csr020__Set_The_Controls_For_the_He
6	Rock__afm020_noctiflora_0407__Neorama	Rock__afm020_noctiflora_0407__Neoran
7	Rock__SCL160__Break_the_Lies__Gnothi_Seauton_R...	Rock__SCL160__Break_the_Lies__Gnothi_Seau
8	Rock__freemusiccharts.songs2012__2012-01-H._Zo...	Rock__freemusiccharts.songs2012__2012-01-
9	Rock__diymAR06__10-Derek_Clegg-Judy	Rock__diymAR06__10-Derek_Clegg-Ju
10	Electronic__NS050__Lake	Electronic__NS050__La
11	Electronic__mtk140__Csanca	Electronic__mtk140__Csan
12	Electronic__foot149__El_And_Eu	Electronic__foot149__El_And_
13	Electronic__nullbomb__spindrumlikeaboom	Electronic__nullbomb__spindrumlikeaboo
14	Electronic__TFR100-VA-TornFleshRecordsPresents...	Electronic__TFR100-VA-TornFleshRecordsPre
15	Electronic__MIXG031__Vozduh	Electronic__MIXG031__Vozdu
16	Electronic__pn037__Sabrina	Electronic__pn037__Sabri
17	Electronic__mtcomp003__latatoo__secede_s_anoth...	Electronic__mtcomp003__latatoo__secede_s_
18	Electronic__ffs104__The_Leather	Electronic__ffs104__The_Leath
19	Electronic__foot018__Baiao_Lo-fi	Electronic__foot018__Baiao_Lo

20 rows × 32 columns

In [8]:

```
# ============================================================
# Cell 8: Evaluate Predictions Against External Manifest
# ============================================================

manifest_eval_df = pd.DataFrame()
manifest_eval_summary_by_genre_df = pd.DataFrame()
manifest_overall_metrics = {}

def labels_intersect(predicted_labels, acceptable_labels):
```

```
    predicted = set(split_pipe_values(predicted_labels))
    acceptable = set(split_pipe_values(acceptable_labels))
    return bool(predicted and acceptable and predicted.intersection(acceptable))


if len(song_summary_df) == 0:
    print("No song predictions to evaluate yet.")
elif "manifest_df" not in globals() or len(manifest_df) == 0:
    print("No manifest loaded. Predictions were produced, but Top-k correctness
cannot be evaluated.")
else:
    ok_summary = song_summary_df[song_summary_df["status"].astype(str).str.lower()
== "ok"].copy()
    if len(ok_summary) == 0:
        print("No successful predictions to evaluate.")
    else:
        manifest_for_merge = manifest_df.copy()
        for col in [
            "file_name",
            "song_title",
            "artist",
            "expected_primary_genre",
            "acceptable_genres",
            "acceptable_labels",
            "source",
            "source_item_url",
            "source_file_url",
            "license_url",
        ]:
            if col not in manifest_for_merge.columns:
                manifest_for_merge[col] = ""

        ok_summary["file_name_key"] =
ok_summary["file_name"].fillna("").astype(str).str.strip().str.lower()
        manifest_for_merge["file_name_key"] =
manifest_for_merge["file_name"].fillna("").astype(str).str.strip().str.lower()

        manifest_cols = [
            "file_name_key",
            "song_title",
            "artist",
            "expected_primary_genre",
            "acceptable_genres",
            "acceptable_labels",
            "source",
            "source_item_url",
            "source_file_url",
            "license_url",
        ]

        manifest_eval_df = ok_summary.merge(
            manifest_for_merge[manifest_cols],
            on="file_name_key",
            how="left",
            suffixes=("", "_manifest"),
        )
```

```
        manifest_eval_df["manifest_matched"] =
manifest_eval_df["acceptable_labels"].fillna("").astype(str).str.len() > 0
        manifest_eval_df["top1_hit"] = manifest_eval_df.apply(
            lambda row: labels_intersect(row.get("top1_labels",
row.get("main_label", "")), row.get("acceptable_labels", "")),
            axis=1,
        )
        manifest_eval_df["top3_hit"] = manifest_eval_df.apply(
            lambda row: labels_intersect(row.get("top3_labels", ""),
row.get("acceptable_labels", "")),
            axis=1,
        )
        manifest_eval_df["top5_hit"] = manifest_eval_df.apply(
            lambda row: labels_intersect(row.get("top5_labels", ""),
row.get("acceptable_labels", "")),
            axis=1,
        )
        manifest_eval_df["stable_prediction_hit"] = manifest_eval_df.apply(
            lambda row: labels_intersect(row.get("stable_predicted_labels", ""),
row.get("acceptable_labels", "")),
            axis=1,
        )
        manifest_eval_df["expected_primary_name_top1_exact"] = (

manifest_eval_df["main_genre_name"].fillna("").astype(str).str.lower().str.strip()
            ==
manifest_eval_df["expected_primary_genre"].fillna("").astype(str).str.lower().str.s
trip()
        )

        eval_base = manifest_eval_df[manifest_eval_df["manifest_matched"]].copy()
        if len(eval_base):
            manifest_eval_summary_by_genre_df =
eval_base.groupby("expected_primary_genre", dropna=False).agg(
                songs=("song_name", "count"),
                top1_hit_rate=("top1_hit", "mean"),
                top3_hit_rate=("top3_hit", "mean"),
                top5_hit_rate=("top5_hit", "mean"),
                stable_prediction_hit_rate=("stable_prediction_hit", "mean"),
                exact_expected_name_top1_rate=("expected_primary_name_top1_exact",
"mean"),
                avg_main_score=("main_score", "mean"),
                low_confidence_rate=("low_confidence", "mean"),
            ).reset_index()

            manifest_overall_metrics = {
                "songs_evaluated_with_manifest": int(len(eval_base)),
                "top1_hit_rate": float(eval_base["top1_hit"].mean()),
                "top3_hit_rate": float(eval_base["top3_hit"].mean()),
                "top5_hit_rate": float(eval_base["top5_hit"].mean()),
                "stable_prediction_hit_rate":
float(eval_base["stable_prediction_hit"].mean()),
                "exact_expected_name_top1_rate":
float(eval_base["expected_primary_name_top1_exact"].mean()),
                "avg_main_score": float(eval_base["main_score"].mean()),
                "low_confidence_rate": float(eval_base["low_confidence"].mean()),
            }
```

```
            print("Manifest-evaluated songs:",
manifest_overall_metrics["songs_evaluated_with_manifest"])
            print("Top-1 hit rate:",
round(manifest_overall_metrics["top1_hit_rate"], 3))
            print("Top-3 hit rate:",
round(manifest_overall_metrics["top3_hit_rate"], 3))
            print("Top-5 hit rate:",
round(manifest_overall_metrics["top5_hit_rate"], 3))
            print("Stable prediction hit rate:",
round(manifest_overall_metrics["stable_prediction_hit_rate"], 3))

            display(manifest_eval_summary_by_genre_df)

            print("\nMiss cases where acceptable genre was not in Top-5:")
            miss_cols = [
                "song_title",
                "artist",
                "expected_primary_genre",
                "acceptable_genres",
                "main_genre_name",
                "main_score",
                "top3_genres",
                "top5_genres",
                "top5_hit",
                "low_confidence",
            ]
            display(eval_base.loc[~eval_base["top5_hit"], [c for c in miss_cols if
c in eval_base.columns]].head(30))
        else:
            print("Predictions ran, but no rows matched the manifest.")
```

```
Manifest-evaluated songs: 60
Top-1 hit rate: 0.217
Top-3 hit rate: 0.667
Top-5 hit rate: 0.833
Stable prediction hit rate: 0.667
```

	expected_primary_genre	songs	top1_hit_rate	top3_hit_rate	top5_hit_rate	stable_prediction
0	Electronic	10	0.2	0.9	0.9	
1	Experimental	10	0.6	1.0	1.0	
2	Folk	10	0.2	0.7	0.7	
3	Hip-Hop	10	0.1	0.4	0.7	
4	Pop	10	0.0	0.0	0.7	
5	Rock	10	0.2	1.0	1.0	

```
Miss cases where acceptable genre was not in Top-5:
```

	song_title	artist	expected_primary_genre	acceptable_genres	main_genre_na
10	Lake	No-Source Netlabel	Electronic	Electronic|Ambient Electronic|IDM|Glitch|Techn...	Experime
30	04 Plusplus - Broken Doors	NaN	Folk	Folk|Singer-Songwriter|Psych-Folk|Freak-Folk|F...	Experime
36	I Hope You're Happy With Me	NaN	Folk	Folk|Singer-Songwriter|Psych-Folk|Freak-Folk|F...	Electr
39	system 1	Various Artists	Folk	Folk|Singer-Songwriter|Psych-Folk|Freak-Folk|F...	Experime
41	Deltitnu EP	Aydio	Hip-Hop	Hip-Hop|Hip-Hop Beats|Alternative Hip-Hop|Rap|...	Experime
45	Morning Mist	Boogie Belgique	Hip-Hop	Hip-Hop|Hip-Hop Beats|Alternative Hip-Hop|Rap|...	R
48	Time2Spare	Anitek	Hip-Hop	Hip-Hop|Hip-Hop Beats|Alternative Hip-Hop|Rap|...	Experime
56	Blue skies	Bjorn Kleinhenz and Pete Thompson	Pop	Pop|Experimental Pop|Synth Pop|Power-Pop	Experime
58	Runaway	NaN	Pop	Pop|Experimental Pop|Synth Pop|Power-Pop	Experime
59	Mark Robinson - Cape May	My Mean Magpie Recordings	Pop	Pop|Experimental Pop|Synth Pop|Power-Pop	Experime

In [9]:

```
# ============================================================
# Cell 9: Save Outputs
# ============================================================

OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
import importlib.util

song_summary_path = OUTPUT_DIR / "notebook50_external_song_summary.csv"
candidate_results_path = OUTPUT_DIR /
"notebook50_external_candidate_scores_topn.csv"
window_results_path = OUTPUT_DIR / "notebook50_external_window_predictions.csv"
error_path = OUTPUT_DIR / "notebook50_external_errors.csv"
manifest_eval_path = OUTPUT_DIR / "notebook50_external_manifest_evaluation.csv"
```

```
manifest_eval_summary_path = OUTPUT_DIR /
"notebook50_external_manifest_evaluation_by_genre.csv"
manifest_eval_xlsx_path = OUTPUT_DIR /
"notebook50_external_manifest_evaluation.xlsx"
json_path = OUTPUT_DIR / "notebook50_external_batch_summary.json"

song_summary_df.to_csv(song_summary_path, index=False)
candidate_results_df.to_csv(candidate_results_path, index=False)
window_results_df.to_csv(window_results_path, index=False)
error_df.to_csv(error_path, index=False)

if "manifest_eval_df" not in globals():
    manifest_eval_df = pd.DataFrame()
if "manifest_eval_summary_by_genre_df" not in globals():
    manifest_eval_summary_by_genre_df = pd.DataFrame()
if "manifest_overall_metrics" not in globals():
    manifest_overall_metrics = {}

manifest_eval_df.to_csv(manifest_eval_path, index=False)
manifest_eval_summary_by_genre_df.to_csv(manifest_eval_summary_path, index=False)

manifest_eval_xlsx_saved = ""
if len(manifest_eval_df):
    if importlib.util.find_spec("openpyxl") is None:
        print("openpyxl is not installed, so the optional Excel evaluation workbook
was skipped. CSV outputs were saved.")
    else:
        try:
            with pd.ExcelWriter(manifest_eval_xlsx_path, engine="openpyxl") as
writer:
                manifest_eval_df.to_excel(writer, sheet_name="Song Evaluation",
index=False)
                manifest_eval_summary_by_genre_df.to_excel(writer, sheet_name="By
Genre", index=False)
                song_summary_df.to_excel(writer, sheet_name="Song Predictions",
index=False)
            manifest_eval_xlsx_saved = str(manifest_eval_xlsx_path)
        except Exception as e:
            print("Could not save Excel evaluation workbook; CSV outputs were
saved.")
            print("Excel writer error:", e)

summary = {
    "created_at": timestamp_now(),
    "notebook": "50_external_song_batch_windowed_inference.ipynb",
    "external_audio_dir": str(EXTERNAL_AUDIO_DIR),
    "output_dir": str(OUTPUT_DIR),
    "songs_attempted": int(len(audio_files)),
    "successful_songs": int((song_summary_df.get("status", pd.Series(dtype=str)) ==
"ok").sum()) if len(song_summary_df) else 0,
    "error_songs": int(len(error_df)),
    "window_seconds": WINDOW_SECONDS,
    "max_windows": MAX_WINDOWS,
    "window_selection_mode": WINDOW_SELECTION_MODE,
    "min_window_votes": MIN_WINDOW_VOTES,
    "stage1_threshold": STAGE1_THRESHOLD,
    "external_manifest_xlsx": str(EXTERNAL_MANIFEST_XLSX_PATH),
```

```
    "external_manifest_csv": str(EXTERNAL_MANIFEST_CSV_PATH),
    "manifest_source_used": str(manifest_path) if "manifest_path" in globals() and
manifest_path is not None else "",
    "manifest_rows": int(len(manifest_df)) if "manifest_df" in globals() else 0,
    "manifest_overall_metrics": manifest_overall_metrics,
    "outputs": {
        "song_summary": str(song_summary_path),
        "candidate_scores_topn": str(candidate_results_path),
        "window_predictions": str(window_results_path),
        "errors": str(error_path),
        "manifest_evaluation": str(manifest_eval_path),
        "manifest_evaluation_by_genre": str(manifest_eval_summary_path),
        "manifest_evaluation_xlsx": manifest_eval_xlsx_saved,
    },
}

with open(json_path, "w", encoding="utf-8") as f:
    json.dump(summary, f, indent=2)

print("Saved outputs:")
for key, value in summary["outputs"].items():
    print(f"{key}: {value}")
print("summary_json:", json_path)
```

```
openpyxl is not installed, so the optional Excel evaluation workbook was skipped. CSV
outputs were saved.
Saved outputs:
song_summary:
E:\SCHOOL\Masters\Capstone_FMA_Project\outputs\notebook50_external_song_batch_windowe
d\notebook50_external_song_summary.csv
candidate_scores_topn:
E:\SCHOOL\Masters\Capstone_FMA_Project\outputs\notebook50_external_song_batch_windowe
d\notebook50_external_candidate_scores_topn.csv
window_predictions:
E:\SCHOOL\Masters\Capstone_FMA_Project\outputs\notebook50_external_song_batch_windowe
d\notebook50_external_window_predictions.csv
errors:
E:\SCHOOL\Masters\Capstone_FMA_Project\outputs\notebook50_external_song_batch_windowe
d\notebook50_external_errors.csv
manifest_evaluation:
E:\SCHOOL\Masters\Capstone_FMA_Project\outputs\notebook50_external_song_batch_windowe
d\notebook50_external_manifest_evaluation.csv
manifest_evaluation_by_genre:
E:\SCHOOL\Masters\Capstone_FMA_Project\outputs\notebook50_external_song_batch_windowe
d\notebook50_external_manifest_evaluation_by_genre.csv
manifest_evaluation_xlsx:
summary_json:
E:\SCHOOL\Masters\Capstone_FMA_Project\outputs\notebook50_external_song_batch_windowe
d\notebook50_external_batch_summary.json
```

In [10]:

```
# ============================================================
# Cell 10: Review Tables
# ============================================================
```

```
print("Song-level predictions:")
if len(song_summary_df):
    review_cols = [
        "song_name",
        "main_genre_name",
        "main_score",
        "main_confidence_level",
        "main_threshold_vote_count",
        "main_top1_window_vote_count",
        "main_top3_window_vote_count",
        "top3_genres",
        "top5_genres",
        "low_confidence",
    ]
    available_cols = [c for c in review_cols if c in song_summary_df.columns]
    display(song_summary_df[available_cols].head(50))
else:
    print("No songs processed yet.")

print("\nTop candidate rows:")
display(candidate_results_df.head(50))

print("\nWindow-level predictions:")
display(window_results_df.head(50))

if len(error_df):
    print("\nErrors:")
    display(error_df)


print("\nManifest evaluation by genre:")
if "manifest_eval_summary_by_genre_df" in globals() and
len(manifest_eval_summary_by_genre_df):
    display(manifest_eval_summary_by_genre_df)
else:
    print("No manifest evaluation summary available.")

print("\nManifest-evaluated song rows:")
if "manifest_eval_df" in globals() and len(manifest_eval_df):
    eval_review_cols = [
        "song_title",
        "artist",
        "expected_primary_genre",
        "main_genre_name",
        "main_score",
        "top1_hit",
        "top3_hit",
        "top5_hit",
        "stable_prediction_hit",
        "top3_genres",
        "top5_genres",
    ]
    display(manifest_eval_df[[c for c in eval_review_cols if c in
manifest_eval_df.columns]].head(60))
else:
    print("No manifest evaluation rows available.")
```

```
```

```
Song-level predictions:
```

	song_name	main_genre_name	main_score	main_con
0	Rock__ca200_cjazz__503._Andrij_Orel_-_Ringo_Qu...	Experimental	0.565439	
1	Rock__afm004_allmyfaults_neonoir__The_course_o...	Experimental	0.578539	
2	Rock__Lethargie.LP__Intro___Im_Herbst	Experimental	0.612928	
3	Rock__Bostaurus.Demo__The_Sky_Full_of_Shadows	Experimental	0.558755	
4	Rock__BS450014__Last_Addiction	Rock	0.481872	Low
5	Rock__csr020__Set_The_Controls_For_the_Heart_o...	Experimental	0.487924	Low
6	Rock__afm020_noctiflora_0407__Neorama	Experimental	0.546284	
7	Rock__SCL160__Break_the_Lies__Gnothi_Seauton_R...	Experimental	0.606244	
8	Rock__freemusiccharts.songs2012__2012-01-H._Zo...	Electronic	0.630960	
9	Rock__diymAR06__10-Derek_Clegg-Judy	Rock	0.390207	Low
10	Electronic__NS050__Lake	Experimental	0.468257	Low
11	Electronic__mtk140__Csanca	Electronic	0.380607	Low
12	Electronic__foot149__El_And_Eu	Rock	0.389795	Low
13	Electronic__nullbomb__spindrumlikeaboom	Electronic	0.536827	
14	Electronic__TFR100-VA-TornFleshRecordsPresents...	Experimental	0.776628	
15	Electronic__MIXG031__Vozduh	Rock	0.456272	Low
16	Electronic__pn037__Sabrina	Experimental	0.561783	
17	Electronic__mtcomp003__latatoo__secede_s_anoth...	Experimental	0.543909	
18	Electronic__ffs104__The_Leather	Experimental	0.562499	
19	Electronic__foot018__Baiao_Lo-fi	Experimental	0.489454	Low
20	Experimental__pcr089EmilDavydov-Sketches__Sket...	Rock	0.439842	Low
21	Experimental__PoodleplayArkestra-TheoryOfColou...	Experimental	0.494070	Low
22	Experimental__ca446_opa__04_-_Russian_State_Mu...	Rock	0.410615	Low
23	Experimental__AlbumInADayVol4__Summer_Breeze	Experimental	0.550025	
24	Experimental__ca350_g__sh-dah	Experimental	0.565486	
25	Experimental__mz001_Mademoi_Selle__Himitsu	Rock	0.519981	
26	Experimental__tpd074__08_Grow_Up_Lamestream_Tu...	Experimental	0.532344	
27	Experimental__ca492_ai__Beautiful_Distraction	Experimental	0.385706	Low

	song_name	main_genre_name	main_score	main_con
28	Experimental__ca300_ca__206_-_Mons_Jacet_-_for...	Experimental	0.702739	
29	Experimental__csr008__Say_Man	Electronic	0.426806	Low
30	Folk__lbn016Plusplus-gameOver__04_Plusplus_-_B...	Experimental	0.434638	Low
31	Folk__lbn007NickRivera-happySongIsAHappySong__...	Experimental	0.368926	Low
32	Folk__lbn017OldSplendifolia-utterlyHeartbreaki...	Experimental	0.462787	Low
33	Folk__EntertainmentForTheBraindead-Roadkillaaa...	Experimental	0.469050	Low
34	Folk__lclcb05DelgarmaDerAndrafolkWorldChanson_...	Rock	0.476159	Low
35	Folk__pm007__The_Treeplanting_Song	Rock	0.402734	Low
36	Folk__pantoufle_tagada__I_Hope_You_re_Happy_Wi...	Electronic	0.687244	
37	Folk__DDW001__02_-_Built_From_Sticks_-_Here_co...	Folk	0.531868	
38	Folk__ca201_meg__My_Orchard_Fire	Folk	0.587839	
39	Folk__ekleipsi23__system_1	Experimental	0.555550	
40	Hip-Hop__DWK123__Soul_Key	Hip-Hop	0.366830	Low
41	Hip-Hop__DWK031__Aydio_-_01_-_Track	Experimental	0.554200	
42	Hip-Hop__dystopiaq029__Times_re_Changin	Electronic	0.465565	Low
43	Hip-Hop__DWK149__Boogieman_Penthouse	Experimental	0.442799	Low
44	Hip-Hop__mia049__lifelong_fiction	Experimental	0.491556	Low
45	Hip-Hop__DWK217__Morning_Mist	Rock	0.394960	Low
46	Hip-Hop__DWK301__Soul_Blue_Tango	Experimental	0.441469	Low
47	Hip-Hop__DWK127__The_Werewolf	Experimental	0.443956	Low
48	Hip-Hop__DWK315__Time2Spare	Experimental	0.620664	
49	Hip-Hop__DWK155__Eazy_Livin	Electronic	0.581886	

```
Top candidate rows:
```

	song_name	audio_
0	Rock__ca200_cjazz__503._Andrij_Orel_-_Ringo_Qu...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
1	Rock__ca200_cjazz__503._Andrij_Orel_-_Ringo_Qu...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
2	Rock__ca200_cjazz__503._Andrij_Orel_-_Ringo_Qu...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
3	Rock__ca200_cjazz__503._Andrij_Orel_-_Ringo_Qu...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
4	Rock__ca200_cjazz__503._Andrij_Orel_-_Ringo_Qu...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
5	Rock__ca200_cjazz__503._Andrij_Orel_-_Ringo_Qu...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
6	Rock__ca200_cjazz__503._Andrij_Orel_-_Ringo_Qu...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
7	Rock__ca200_cjazz__503._Andrij_Orel_-_Ringo_Qu...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
8	Rock__ca200_cjazz__503._Andrij_Orel_-_Ringo_Qu...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
9	Rock__ca200_cjazz__503._Andrij_Orel_-_Ringo_Qu...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
10	Rock__ca200_cjazz__503._Andrij_Orel_-_Ringo_Qu...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
11	Rock__ca200_cjazz__503._Andrij_Orel_-_Ringo_Qu...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
12	Rock__ca200_cjazz__503._Andrij_Orel_-_Ringo_Qu...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
13	Rock__ca200_cjazz__503._Andrij_Orel_-_Ringo_Qu...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
14	Rock__ca200_cjazz__503._Andrij_Orel_-_Ringo_Qu...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
15	Rock__afm004_allmyfaults_neonoir__The_course_o...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
16	Rock__afm004_allmyfaults_neonoir__The_course_o...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
17	Rock__afm004_allmyfaults_neonoir__The_course_o...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
18	Rock__afm004_allmyfaults_neonoir__The_course_o...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
19	Rock__afm004_allmyfaults_neonoir__The_course_o...	E:\SCHOOL\Masters\Capstone_FMA_Project\data

	song_name	audio_
20	Rock__afm004_allmyfaults_neonoir__The_course_o...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
21	Rock__afm004_allmyfaults_neonoir__The_course_o...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
22	Rock__afm004_allmyfaults_neonoir__The_course_o...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
23	Rock__afm004_allmyfaults_neonoir__The_course_o...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
24	Rock__afm004_allmyfaults_neonoir__The_course_o...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
25	Rock__afm004_allmyfaults_neonoir__The_course_o...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
26	Rock__afm004_allmyfaults_neonoir__The_course_o...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
27	Rock__afm004_allmyfaults_neonoir__The_course_o...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
28	Rock__afm004_allmyfaults_neonoir__The_course_o...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
29	Rock__afm004_allmyfaults_neonoir__The_course_o...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
30	Rock__Lethargie.LP__Intro___Im_Herbst	E:\SCHOOL\Masters\Capstone_FMA_Project\data
31	Rock__Lethargie.LP__Intro___Im_Herbst	E:\SCHOOL\Masters\Capstone_FMA_Project\data
32	Rock__Lethargie.LP__Intro___Im_Herbst	E:\SCHOOL\Masters\Capstone_FMA_Project\data
33	Rock__Lethargie.LP__Intro___Im_Herbst	E:\SCHOOL\Masters\Capstone_FMA_Project\data
34	Rock__Lethargie.LP__Intro___Im_Herbst	E:\SCHOOL\Masters\Capstone_FMA_Project\data
35	Rock__Lethargie.LP__Intro___Im_Herbst	E:\SCHOOL\Masters\Capstone_FMA_Project\data
36	Rock__Lethargie.LP__Intro___Im_Herbst	E:\SCHOOL\Masters\Capstone_FMA_Project\data
37	Rock__Lethargie.LP__Intro___Im_Herbst	E:\SCHOOL\Masters\Capstone_FMA_Project\data
38	Rock__Lethargie.LP__Intro___Im_Herbst	E:\SCHOOL\Masters\Capstone_FMA_Project\data
39	Rock__Lethargie.LP__Intro___Im_Herbst	E:\SCHOOL\Masters\Capstone_FMA_Project\data

	song_name	audio_
40	Rock__Lethargie.LP__Intro___Im_Herbst	E:\SCHOOL\Masters\Capstone_FMA_Project\data
41	Rock__Lethargie.LP__Intro___Im_Herbst	E:\SCHOOL\Masters\Capstone_FMA_Project\data
42	Rock__Lethargie.LP__Intro___Im_Herbst	E:\SCHOOL\Masters\Capstone_FMA_Project\data
43	Rock__Lethargie.LP__Intro___Im_Herbst	E:\SCHOOL\Masters\Capstone_FMA_Project\data
44	Rock__Lethargie.LP__Intro___Im_Herbst	E:\SCHOOL\Masters\Capstone_FMA_Project\data
45	Rock__Bostaurus.Demo__The_Sky_Full_of_Shadows	E:\SCHOOL\Masters\Capstone_FMA_Project\data
46	Rock__Bostaurus.Demo__The_Sky_Full_of_Shadows	E:\SCHOOL\Masters\Capstone_FMA_Project\data
47	Rock__Bostaurus.Demo__The_Sky_Full_of_Shadows	E:\SCHOOL\Masters\Capstone_FMA_Project\data
48	Rock__Bostaurus.Demo__The_Sky_Full_of_Shadows	E:\SCHOOL\Masters\Capstone_FMA_Project\data
49	Rock__Bostaurus.Demo__The_Sky_Full_of_Shadows	E:\SCHOOL\Masters\Capstone_FMA_Project\data

```
Window-level predictions:
```

	song_name	audio_
0	Rock__ca200_cjazz__503._Andrij_Orel_-_Ringo_Qu...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
1	Rock__ca200_cjazz__503._Andrij_Orel_-_Ringo_Qu...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
2	Rock__ca200_cjazz__503._Andrij_Orel_-_Ringo_Qu...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
3	Rock__ca200_cjazz__503._Andrij_Orel_-_Ringo_Qu...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
4	Rock__ca200_cjazz__503._Andrij_Orel_-_Ringo_Qu...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
5	Rock__ca200_cjazz__503._Andrij_Orel_-_Ringo_Qu...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
6	Rock__ca200_cjazz__503._Andrij_Orel_-_Ringo_Qu...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
7	Rock__ca200_cjazz__503._Andrij_Orel_-_Ringo_Qu...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
8	Rock__afm004_allmyfaults_neonoir__The_course_o...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
9	Rock__afm004_allmyfaults_neonoir__The_course_o...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
10	Rock__afm004_allmyfaults_neonoir__The_course_o...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
11	Rock__afm004_allmyfaults_neonoir__The_course_o...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
12	Rock__afm004_allmyfaults_neonoir__The_course_o...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
13	Rock__afm004_allmyfaults_neonoir__The_course_o...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
14	Rock__afm004_allmyfaults_neonoir__The_course_o...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
15	Rock__afm004_allmyfaults_neonoir__The_course_o...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
16	Rock__Lethargie.LP__Intro___Im_Herbst	E:\SCHOOL\Masters\Capstone_FMA_Project\data
17	Rock__Lethargie.LP__Intro___Im_Herbst	E:\SCHOOL\Masters\Capstone_FMA_Project\data
18	Rock__Lethargie.LP__Intro___Im_Herbst	E:\SCHOOL\Masters\Capstone_FMA_Project\data
19	Rock__Lethargie.LP__Intro___Im_Herbst	E:\SCHOOL\Masters\Capstone_FMA_Project\data
20	Rock__Lethargie.LP__Intro___Im_Herbst	E:\SCHOOL\Masters\Capstone_FMA_Project\data
21	Rock__Lethargie.LP__Intro___Im_Herbst	E:\SCHOOL\Masters\Capstone_FMA_Project\data
22	Rock__Lethargie.LP__Intro___Im_Herbst	E:\SCHOOL\Masters\Capstone_FMA_Project\data
23	Rock__Lethargie.LP__Intro___Im_Herbst	E:\SCHOOL\Masters\Capstone_FMA_Project\data
24	Rock__Bostaurus.Demo__The_Sky_Full_of_Shadows	E:\SCHOOL\Masters\Capstone_FMA_Project\data
25	Rock__Bostaurus.Demo__The_Sky_Full_of_Shadows	E:\SCHOOL\Masters\Capstone_FMA_Project\data
26	Rock__Bostaurus.Demo__The_Sky_Full_of_Shadows	E:\SCHOOL\Masters\Capstone_FMA_Project\data
27	Rock__Bostaurus.Demo__The_Sky_Full_of_Shadows	E:\SCHOOL\Masters\Capstone_FMA_Project\data
28	Rock__Bostaurus.Demo__The_Sky_Full_of_Shadows	E:\SCHOOL\Masters\Capstone_FMA_Project\data
29	Rock__Bostaurus.Demo__The_Sky_Full_of_Shadows	E:\SCHOOL\Masters\Capstone_FMA_Project\data
30	Rock__Bostaurus.Demo__The_Sky_Full_of_Shadows	E:\SCHOOL\Masters\Capstone_FMA_Project\data

	song_name	audio_
31	Rock__Bostaurus.Demo__The_Sky_Full_of_Shadows	E:\SCHOOL\Masters\Capstone_FMA_Project\data
32	Rock__BS450014__Last_Addiction	E:\SCHOOL\Masters\Capstone_FMA_Project\data
33	Rock__BS450014__Last_Addiction	E:\SCHOOL\Masters\Capstone_FMA_Project\data
34	Rock__BS450014__Last_Addiction	E:\SCHOOL\Masters\Capstone_FMA_Project\data
35	Rock__BS450014__Last_Addiction	E:\SCHOOL\Masters\Capstone_FMA_Project\data
36	Rock__BS450014__Last_Addiction	E:\SCHOOL\Masters\Capstone_FMA_Project\data
37	Rock__BS450014__Last_Addiction	E:\SCHOOL\Masters\Capstone_FMA_Project\data
38	Rock__BS450014__Last_Addiction	E:\SCHOOL\Masters\Capstone_FMA_Project\data
39	Rock__BS450014__Last_Addiction	E:\SCHOOL\Masters\Capstone_FMA_Project\data
40	Rock__csr020__Set_The_Controls_For_the_Heart_o...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
41	Rock__csr020__Set_The_Controls_For_the_Heart_o...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
42	Rock__csr020__Set_The_Controls_For_the_Heart_o...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
43	Rock__csr020__Set_The_Controls_For_the_Heart_o...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
44	Rock__csr020__Set_The_Controls_For_the_Heart_o...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
45	Rock__csr020__Set_The_Controls_For_the_Heart_o...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
46	Rock__csr020__Set_The_Controls_For_the_Heart_o...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
47	Rock__csr020__Set_The_Controls_For_the_Heart_o...	E:\SCHOOL\Masters\Capstone_FMA_Project\data
48	Rock__afm020_noctiflora_0407__Neorama	E:\SCHOOL\Masters\Capstone_FMA_Project\data
49	Rock__afm020_noctiflora_0407__Neorama	E:\SCHOOL\Masters\Capstone_FMA_Project\data

```
Manifest evaluation by genre:
```

	expected_primary_genre	songs	top1_hit_rate	top3_hit_rate	top5_hit_rate	stable_prediction
0	Electronic	10	0.2	0.9	0.9	
1	Experimental	10	0.6	1.0	1.0	
2	Folk	10	0.2	0.7	0.7	
3	Hip-Hop	10	0.1	0.4	0.7	
4	Pop	10	0.0	0.0	0.7	
5	Rock	10	0.2	1.0	1.0	

```
Manifest-evaluated song rows:
```

	song_title	artist	expected_primary_genre	main_genre_name	main_score	t
0	503. Andrij Orel - Ringo Quartz	Various	Rock	Experimental	0.565439	
1	The course of true love never did run smooth, ...	all:my:faults	Rock	Experimental	0.578539	
2	Intro / Im Herbst	Lethargie	Rock	Experimental	0.612928	
3	The Sky Full of Shadows	Bostaurus	Rock	Experimental	0.558755	
4	Last Addiction	Pete Lund	Rock	Rock	0.481872	
5	Set The Controls For the Heart of the Sun	Various Artists	Rock	Experimental	0.487924	
6	Neorama	Noctiflora	Rock	Experimental	0.546284	
7	Break the Lies (Gnothi Seauton Remix)	Tigerberry	Rock	Experimental	0.606244	
8	2012-01-H. Zombie-Taxi Driver	NaN	Rock	Electronic	0.630960	
9	Across Town	Derek Clegg	Rock	Rock	0.390207	
10	Lake	No-Source Netlabel	Electronic	Experimental	0.468257	
11	Csanca	ST	Electronic	Electronic	0.380607	
12	El And Eu	Edmar Travassos	Electronic	Rock	0.389795	
13	Nullbomb creatures 2005-2006	NaN	Electronic	Electronic	0.536827	
14	36 - Vulgar Disease - Every Noise Has Its TORN	Torn Flesh Records	Electronic	Experimental	0.776628	
15	Vozduh	NaN	Electronic	Rock	0.456272	
16	Sabrina	Francisco Pinto	Electronic	Experimental	0.561783	
17	latatoo (secede's another day wasted mix)	NaN	Electronic	Experimental	0.543909	

	song_title	artist	expected_primary_genre	main_genre_name	main_score	t
18	The Leather	Antuan Graftio	Electronic	Experimental	0.562499	
19	Baiao Lo-fi	Chico Correa & Electronic Band	Electronic	Experimental	0.489454	
20	Sketch No.3	E. Davydov	Experimental	Rock	0.439842	
21	Theory Of Colour	Poodleplay Arkestra	Experimental	Experimental	0.494070	
22	04 - Russian State Museum's Lottery	Opa!	Experimental	Rock	0.410615	
23	Summer Breeze	various artists	Experimental	Experimental	0.550025	
24	sh-dah	gillicuddy	Experimental	Experimental	0.565486	
25	Himitsu	Mademoi Selle	Experimental	Rock	0.519981	
26	Take Pills - 06.08.08 Happy Homeless [tpd074]	Andrew Cauthen	Experimental	Experimental	0.532344	
27	Beautiful Distraction	Arcade Island	Experimental	Experimental	0.385706	
28	206 - Mons Jacet - for spring	Various	Experimental	Experimental	0.702739	
29	Say Man	R. Stevie Moore	Experimental	Electronic	0.426806	
30	04 Plusplus - Broken Doors	NaN	Folk	Experimental	0.434638	
31	01 ReneeLuise	Nick Rivera	Folk	Experimental	0.368926	
32	Two Eagles	NaN	Folk	Experimental	0.462787	
33	Relapse	Julia Kotowski	Folk	Experimental	0.469050	
34	LCLCB05_-_Delgarma_-_Der_Andra_-_05_-_You can ...	NaN	Folk	Rock	0.476159	
35	The Treeplanting Song	PK	Folk	Rock	0.402734	
36	I Hope You're Happy With Me	NaN	Folk	Electronic	0.687244	
37	Built From Sticks - Half	NaN	Folk	Folk	0.531868	

	song_title	artist	expected_primary_genre	main_genre_name	main_score	t
	Steps					
38	My Orchard Fire	Menhirs of Er Grah	Folk	Folk	0.587839	
39	system 1	Various Artists	Folk	Experimental	0.555550	
40	Soul Key	ProleteR	Hip-Hop	Hip-Hop	0.366830	
41	Deltitnu EP	Aydio	Hip-Hop	Experimental	0.554200	
42	Times're Changin'	Various Artists	Hip-Hop	Electronic	0.465565	
43	Boogieman Penthouse	Boogie Belgique	Hip-Hop	Experimental	0.442799	
44	lifelong fiction	NaN	Hip-Hop	Experimental	0.491556	
45	Morning Mist	Boogie Belgique	Hip-Hop	Rock	0.394960	
46	Soul Blue Tango	Mounika	Hip-Hop	Experimental	0.441469	
47	The Werewolf	Kova	Hip-Hop	Experimental	0.443956	
48	Time2Spare	Anitek	Hip-Hop	Experimental	0.620664	
49	Eazy Livin'	Poldoore	Hip-Hop	Electronic	0.581886	
50	01 - Monochromatic - Immobility-	MonoChromatic	Pop	Electronic	0.424461	
51	05 - Doing Time	Garmisch	Pop	Experimental	0.504474	
52	Closer to You (feat. SHA)	starfrosch	Pop	Electronic	0.659494	
53	Heaven In Your Mouth	intouch	Pop	Experimental	0.558473	
54	Falling	Twizzle	Pop	Experimental	0.520471	
55	The Virus Left Behind	James Gardner	Pop	Experimental	0.545824	
56	Blue skies	Bjorn Kleinhenz and Pete Thompson	Pop	Experimental	0.551967	
57	09 - Universal Materials - This is the Macrophage	Michael Gregoire	Pop	Experimental	0.485410	
58	Runaway	NaN	Pop	Experimental	0.524945	

	song_title	artist	expected_primary_genre	main_genre_name	main_score	t
59	Mark Robinson - Cape May	My Mean Magpie Recordings	Pop	Experimental	0.498985	

How to Report Notebook 50

Notebook 50 should be described as a deployment-style external-audio experiment, not the formal benchmark.

Suggested wording:

> Notebook 50 applies the trained audio candidate-150 model to complete external songs. Each song is split into multiple 15-second windows, each window is scored by the model, and song-level predictions are produced by averaging genre confidence across windows while also reporting how often each genre appears across the windows. This supports a recommendation-style interpretation using Top-1, Top-3, and Top-5 candidate genres.

Important caveat:

- Notebook 49 is the formal held-out FMA evaluation.
- Notebook 50 is a practical deployment test for complete songs and may experience domain shift from commercial/external audio.

With the external manifest enabled, report Top-1, Top-3, and Top-5 hit rates against the `acceptable_labels` column. This is stronger than exact-name matching because many songs have valid subgenre/parent genre alternatives.